

✓ Course: Data Analytics Using Python (AII0202)

Module 2 – Activity 2.1

NumPy Hands-on Lab (5 Marks)

Instructions:

- Complete all tasks.
- Run each cell and show outputs.
- Add comments wherever required.
- Submit the completed notebook(PDF) on LMS.

Name:Anshika khandelwal

Roll No.:BETN1AI25001

colab link:<https://colab.research.google.com/drive/16WkYBRBXVHHBX7fkbSLmMfOcDwHKuuAm#scrollTo=qFDBmFqcB3dV>

```
# Import NumPy
import numpy as np
print('NumPy Version:', np.__version__)
```

```
NumPy Version: 2.0.2
```

✓ Task 1: Array Creation(1 Mark)

Create:

1D array of 10 integers

2D array (3×3 matrix)

Identity matrix of size 4

Random array (5×5)

```
# Create required arrays here

# 1D array of 10 integers
import numpy as np
print("1D array of 10 integers:")
a=np.arange(10)
print(a)
print("\n")

# 3x3 matrix
print("3x3 matrix is:")
b=np.array([[1,2,3],[4,5,6],[7,8,9]])
print(b)
print("\n")

# Identity matrix of size 4
identity_matrix = np.identity(4)
print("Identity Matrix (4x4):\n", identity_matrix)
print("\n")

# Random 5x5 matrix
print("random array of 5x5 is :")
d=np.random.rand(5,5)
print(d)
```

```
1D array of 10 integers:
[0 1 2 3 4 5 6 7 8 9]
```

```
3x3 matrix is:
[[1 2 3]
```

```
[4 5 6]
[7 8 9]]

Identity Matrix (4x4):
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

random array of 5x5 is :
[[0.5005364  0.29048047 0.3984345  0.23037631 0.51750072]
 [0.40886052 0.30857137 0.93141749 0.26266592 0.35139264]
 [0.58942973 0.82872604 0.06303743 0.28754938 0.43716892]
 [0.89929326 0.33549185 0.1629068  0.57939404 0.56793117]
 [0.53410449 0.99252964 0.55710091 0.12424011 0.64742669]]
```

Task 2: Indexing & Slicing (1 Mark)

Given:

A = np.arange(1, 26).reshape(5,5)

Perform:

Extract 2nd row

Extract 3rd column

Extract 2x2 submatrix (center)

Reverse last row

```
A = np.arange(1, 26).reshape(5,5)
print(A)
print("\n")

# Extract 2nd row
print("2nd row of matrix is :")
print(A[1,...])
print("\n")

# Extract 3rd column
print("thrid column of matrix is ")
print(A[:,2])
print("\n")

# Extract 2x2 center matrix
print("2x2 center matrix is :")
center_submatrix = A[1:3, 1:3]
print(center_submatrix)
print("\n")

# Reverse last row
reverse_last_row = A[-1][::-1]
print("Reversed Last Row:", reverse_last_row)
```

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]]
```

```
2nd row of matrix is :
[ 6  7  8  9 10]
```

```
thrid column of matrix is
[ 3  8 13 18 23]
```

```
2x2 center matrix is :
[[ 7  8]
 [12 13]]
```

```
Reversed Last Row: [25 24 23 22 21]
```

Task 3: Broadcasting(1 Mark)

Perform broadcasting operations on matrix A created in Task-2.

Add scalar 10 to matrix A

Add vector [1,2,3,4,5] to each row

Normalize matrix values between 0–1

```
# Perform broadcasting operations here
import numpy as np

A = np.arange(1, 26).reshape(5,5)
print("Matrix A:\n", A)
# Add scalar 10 to matrix
A_scalar = A + 10
print("After Adding 10:\n", A_scalar)
# Add vector [1,2,3,4,5] to each row
vector = np.array([1,2,3,4,5])
A_vector = A + vector
print("After Adding Vector to Each Row:\n", A_vector)
# Normalize matrix between 0 and 1
```

```
Matrix A:
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]]
After Adding 10:
[[11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]
 [26 27 28 29 30]
 [31 32 33 34 35]]
After Adding Vector to Each Row:
[[ 2  4  6  8 10]
 [ 7  9 11 13 15]
 [12 14 16 18 20]
 [17 19 21 23 25]
 [22 24 26 28 30]]
```

Task 4: Vectorized Computation(1 Marks)

Perform element-wise multiplication Compute dot product Compute mean, standard deviation

```
X = np.array([1,2,3,4])
Y = np.array([5,6,7,8])

# Element-wise multiplication
print("element wise multiplication is :",X*Y)

# Dot product
dot_product = np.dot(X, Y)
print("Dot Product:", dot_product)

# Mean and Standard Deviation
mean_X = np.mean(X)
mean_Y = np.mean(Y)

print("Mean of X:", mean_X)
print("Mean of Y:", mean_Y)

std_X = np.std(X)
std_Y = np.std(Y)

print("Standard Deviation of X:", std_X)
print("Standard Deviation of Y:", std_Y)
```

```
element wise multiplication is : [ 5 12 21 32]
Dot Product: 70
```

```
Mean of X: 2.5
Mean of Y: 6.5
Standard Deviation of X: 1.118033988749895
Standard Deviation of Y: 1.118033988749895
```

Task 5: Mini Case Problem (1 Mark)

Dataset: Student marks (5 students, 3 subjects)

Compute total and average per student

Find topper

Find subject-wise highest score

```
marks = np.array([[80,75,90],[60,88,70],[95,92,85],[70,65,60],[88,78,84]])
# Compute totals, averages, topper
total_per_student = np.sum(marks,axis=1)

average=np.mean(marks,axis=1)

print("Total marks per student:", total_per_student)
print("average per student:",average)
print("\n")

topper_index = np.argmax(total_per_student)
print("Topper is Student", topper_index + 1)
print("Topper Total Marks:", total_per_student[topper_index])
print("\n")

subject_highest = np.max(marks, axis=0)
print("Subject-wise highest scores:", subject_highest)
```

Total marks per student: [245 218 272 195 250]
average per student: [81.66666667 72.66666667 90.66666667 65. 83.33333333]

Topper is Student 3
Topper Total Marks: 272

Subject-wise highest scores: [95 92 90]